Northern University of Business and Technology Khulna
Department of Computer Science and Engineering

WEEK 12: FINAL PROJECT REPORT

Project Title: AI-Powered Resume Analyzer and Career Copilot (RESUMIND)

Course Title: Mobile Computing Lab

Course Code: CSE4204

Submitted to:
Md. Riaz Mahmud
Assistant Professor
Department of Computer Science and Engineering
Northern University of Business and Technology, Khulna

Submission Date: 22 October 2026

TEAM INFORMATION SHEET

Field	Details
Official Team Name	CSE4204-8A-T05
Section	8A
Project Title	AI-Powered Resume Analyzer and Career Copilot (RESUMIND)
Team Member List with IDs	1. Mahammad Hasan (ID: 11220320828) - Team Leader / Frontend Developer + UI/UX Designer 2. Shanawaz Sakib (ID: 11220320916) - Core System Integration Engineer 3. Ashikur Rahman Himel (ID: 11220320833) - Database Storage & Quality Controller 4. Rasel Ratul (ID: 11220320827) - Backend Developer
Live Application Link	https://ai-resume-analyzer-psi-lime.vercel.app
GitHub Repository Link	https://github.com/mahamud894/AI_Resume_Analyzer
Figma Link	https://www.figma.com/design/Ls0B3DIjxRKrP2RIzYQxPG/AI-Resume-Analyzer?node-id=2-2&t=lFEVQCoUHt0JSumI-1

Abstract

Many capable students and fresh graduates are rejected before a human ever reads their resume, because an Applicant Tracking System (ATS) filters it out for weak keywords, unclear structure or poor formatting. **RESUMIND** is a responsive web application that lets a job seeker upload a PDF resume, paste a job description and receive an AI-generated evaluation within about 20 seconds: an overall score, five category scores (ATS, tone & style, content, structure, skills) and concrete tips.

Beyond analysis, RESUMIND rewrites the resume into the **European (Europass) CV standard** without inventing any facts, measures the improvement with a **before / after score**, and generates a matching **one-page cover letter**. The system is built with React 19 and React Router 8 on the frontend, an Express 5 REST API on the backend, MongoDB Atlas (with GridFS for files) as the database, and Google Gemini in structured-JSON mode as the AI engine. It is deployed on Vercel (frontend) and Render (backend), protected by bcrypt + JWT authentication and optional Google Sign-In, and verified by 48 automated API tests that run in GitHub Actions on every push.

Live application	https://ai-resume-analyzer-psi-lime.vercel.app
Backend API	https://resumind-api-3kpd.onrender.com/api/health
Source code	https://github.com/mahamud894/AI_Resume_Analyzer

1. Introduction

Recruitment today is largely automated. Large employers and many local companies receive hundreds of applications for one position and use ATS software to rank them before a recruiter looks at the list. A resume that is well-written for a human can still score poorly in such a system if it lacks the job's keywords, uses an unusual layout or hides important information.

RESUMIND ("AI-Powered Resume Analyzer and Career Copilot") was developed by team CSE4204-8A-T05 as the term project of the Mobile Computing Lab. It acts as a personal career assistant that works on any device: the user signs in, uploads a resume, describes the job, and receives structured, explainable feedback. The application then helps the user act on that feedback by producing an improved Europass CV and a cover letter that are both tied to the same job.

1.1 Scope

- Secure account management (email/password and Google Sign-In, change password, delete account).
- PDF resume upload with a page-1 preview, stored in MongoDB GridFS.
- AI analysis of a resume against a specific job title and description.
- AI rewrite into a Europass CV with an editable, printable A4 layout and a list of every change.
- Before / after score comparison and a score-history chart.
- AI cover letter generation (formal or modern tone) with inline editing and PDF download.
- A history dashboard of all resumes and analyses, and a printable report.
- Responsive design for mobile, tablet and desktop; deployment to the public internet.

1.2 Document structure

Chapter 2 states the problem and Chapter 3 the objectives. Chapter 4 explains the methodology, Chapter 5 the architecture and Chapter 6 the database design. Chapter 7 describes the AI integration and Chapter 8 security and authentication. Chapter 9 presents the implementation, Chapter 10 testing and Chapter 11 the results. Chapter 12 covers deployment, Chapter 13 limitations and future work, and Chapter 14 concludes. The individual contributions are listed in Chapter 15.

2. Problem Statement

2.1 The existing problem

Modern recruitment relies heavily on Applicant Tracking Systems. Capable candidates — especially fresh graduates in Bangladesh who are applying to international or remote positions — are frequently filtered out automatically because of poor formatting, weak action verbs or missing keywords from the job description. They receive repeated rejections without knowing what to fix.

2.2 Why the problem matters

- Job hunting is highly competitive; a single missed keyword can remove an otherwise qualified applicant.
- Writing a personalised cover letter for every application is time-consuming, so many applicants send generic letters.

- European and international employers expect the **Europass** CV format (MM/YYYY dates, CEFR language levels, EQF qualification levels), which most local graduates have never been taught.

2.3 Why existing solutions are insufficient

Table 1: Comparison with existing tools

Tool	What it does well	Gap that RESUMIND addresses
Jobscan	Keyword match rate against a job description	Paid after a few scans; keyword matching without contextual explanation; no Europass rewrite
Resume Worded	Line-by-line resume feedback	Generic scoring not tied to one job; cover letters and full rewrite are premium
Reactive Resume	Free, open-source resume builder	Builder only — no AI analysis, no ATS score, no cover letter tied to the job
Europass online editor	Official EU CV template	Manual data entry; no feedback on content quality or job fit

2.4 Why AI is relevant

A rule-based keyword counter cannot judge whether a bullet point demonstrates a skill or whether the tone is professional. A large language model can read the resume and the job description in context, score each dimension, explain its reasoning in plain language and rewrite content — as long as its output is constrained so that it stays structured, consistent and truthful. RESUMIND uses Google Gemini with a strict JSON schema and code-level post-checks to achieve exactly that.

3. Objectives

The original proposal (Week 02) defined the objectives below. During development the team replaced the client-only Puter.js approach with its own backend so that the system matches the course ERD and the Week 06 backend requirements; the objectives were updated accordingly.

Table 2: Project objectives and how they were met

#	Objective	How it is achieved	Status
O1	Secure authentication and session management	bcrypt password hashing, 7-day JWT, Google Sign-In, token invalidation on password change, rate limiting	Done

#	Objective	How it is achieved	Status
O2	Document vault with history	PDF + preview image in MongoDB GridFS; dashboard lists every resume with its analyses	Done
O3	AI-driven ATS analysis against a job	Gemini JSON mode with a fixed schema: overall score + 5 categories with tips	Done
O4	Actionable improvement	Europass CV rewrite that addresses every "improve" tip, never invents facts, lists each change	Done
O5	Measure the improvement	Re-score the improved CV for the same job; before/after comparison and history chart	Done
O6	Personalised cover letter	European one-page business letter generated only from CV facts; editable and printable	Done
O7	Responsive, mobile-friendly UI	Tailwind CSS 4 mobile-first layout; tested at phone, tablet and desktop widths	Done
O8	Quality and deployment	48 automated API tests in CI; live on Vercel + Render + Atlas	Done

4. Methodology

4.1 Development process

The team followed an **iterative, incremental** process aligned with the weekly lab schedule. Each week produced a working increment that was reviewed by the course teacher, and feedback was folded into the next iteration. GitHub was used for version control, with every change pushed to the `main` branch and verified by continuous integration.

Table 3: Weekly development timeline

Week	Phase	Main outcome
02	Proposal	Problem, objectives, feasibility, similar systems, team roles
03	Requirements (SRS)	Functional / non-functional requirements, use-case diagram, first ERD
04	System design	Architecture, activity diagram, refined ERD
05	UI / UX	Wireframes and design planning (RESUMIND visual identity)
06	Backend & database	Express API, MongoDB models, JWT authentication
07	Frontend integration	React pages connected to the REST API
08	AI integration	Gemini analysis with structured JSON output
09	Feature completion	Europass CV, cover letter, account management
10	Testing	Vitest + Supertest suite, testing report

Week	Phase	Main outcome
11	Deployment	Vercel + Render + MongoDB Atlas, CI on GitHub Actions
12–14	Documentation & presentation	This report, user manual, API documentation, slides, viva

4.2 From Puter.js to an own backend

The first prototype (Week 07) followed a tutorial and ran entirely in the browser using the Puter.js SDK for login, file storage, key-value data and AI. This was quick to build but did not satisfy the course requirements for a backend, a database that matches the ERD, or control over security and validation. The team therefore built its own Express + MongoDB backend and moved the AI calls to the server, where the API key is hidden and the output can be validated.

Table 4: Migration from the prototype to the final system

Concern	Prototype (Puter.js)	Final system
Login	puter.auth.signIn()	POST /api/auth/login, /register, /google (bcrypt + JWT)
File storage	puter.fs	MongoDB GridFS (uploads.files / uploads.chunks)
Data	puter.kv key-value store	MongoDB collections that match the ERD
AI	puter.ai in the browser	Gemini on the server, JSON schema, retries, post-checks
Security	Handled by Puter	helmet, CORS allow-list, input validation, rate limits

4.3 Tools and technologies

Table 5: Technology stack

Layer	Technology	Why it was chosen
Frontend	React 19, React Router 8, TypeScript, Tailwind CSS 4, Zustand, pdf.js	Component model, type safety, fast mobile-first styling, light global state, page-1 PDF preview in the browser
Backend	Node.js, Express 5, Multer, pdf-parse, express-validator, helmet	Same language as the frontend, minimal and well-documented, mature security middleware
Database	MongoDB Atlas, Mongoose 9, GridFS	Flexible documents fit nested AI feedback and CVs; free managed cluster; files stored with the data

Layer	Technology	Why it was chosen
AI	Google Gemini via @google/genai (JSON mode)	Free tier, long context, structured output with a JSON schema
Auth	bcryptjs, jsonwebtoken, google-auth-library	Industry-standard password hashing, stateless tokens, verified Google ID tokens
Testing / CI	Vitest, Supertest, mongodb-memory-server, GitHub Actions	Real HTTP tests against an in-memory database; runs on every push
Hosting	Vercel (frontend), Render (backend), Docker Compose (local)	Free tiers with automatic deploys from GitHub

5. System Architecture

RESUMIND uses a classic **three-tier client–server architecture**. The browser runs a single-page React application; it talks only to the Express REST API over HTTPS using JSON and a JWT bearer token. The API is the only component that can reach the database and the Gemini API, so secrets never leave the server.

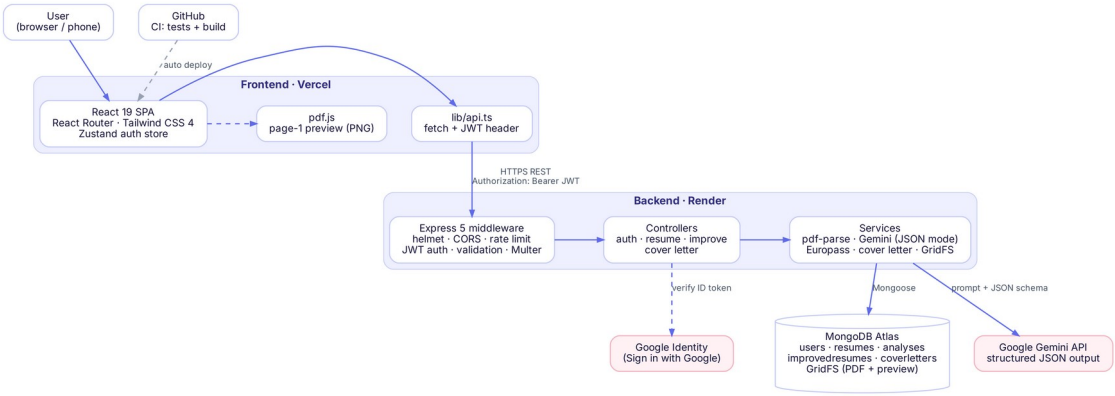


Figure 1: Three-tier deployment architecture

5.1 Components

- **Presentation tier (Vercel):** React SPA — landing page, auth, dashboard, upload, result, Europass CV, cover letter and account pages. pdf.js renders a preview image of page 1 before upload.
- **Application tier (Render):** Express 5 API organised as routes → middleware (auth, validation, upload, rate limit) → controllers → services (pdf, Gemini, Europass, cover letter).
- **Data tier (MongoDB Atlas):** five collections plus GridFS buckets for the uploaded PDFs and preview images.

- **External services:** Google Gemini (analysis, rewrite, scoring, cover letter) and Google Identity Services (Sign-In).

5.2 Backend structure

```
backend/
├── server.js            entry point: connect to MongoDB, start HTTP server
└── src/
    ├── app.js           helmet, CORS, JSON parser, routes, error handler
    ├── config/          env.js (validated settings), db.js
    ├── models/          User, Resume, Analysis, ImprovedResume, CoverLetter
    ├── routes/          authRoutes.js, resumeRoutes.js
    ├── middleware/      auth.js (JWT), upload.js (Multer), validate.js,
errorHandler.js
    ├── controllers/     auth, resume, improve, coverLetter
    ├── services/        pdfService, geminiService, europassService,
coverLetterService
    └── utils/           ApiError, token
```

5.3 Request lifecycle (analysis example)

1. The user submits the upload form; the frontend sends `POST /api/resume/upload` (multipart) with the JWT.
2. Multer checks the type and size and verifies that the bytes start with `%PDF-`; the files go to GridFS.
3. The frontend sends `POST /api/resume/analyze` with the resume id and job details.
4. `requireAuth` verifies the token; express-validator checks the input; the AI rate limiter checks the user's hourly quota.
5. The controller extracts text with pdf-parse, calls Gemini in JSON mode, normalises the result and saves an `Analysis`.
6. The API returns `201 { analysis }`; the frontend navigates to the result page and renders the scores.

6. Database Design

MongoDB was chosen because the main entities contain deeply nested, variable-length data (AI feedback with tip arrays, a full Europass CV, a structured letter). Each entity is still a separate collection with explicit references, so the design maps one-to-one to the ERD defined in Weeks 03–04.

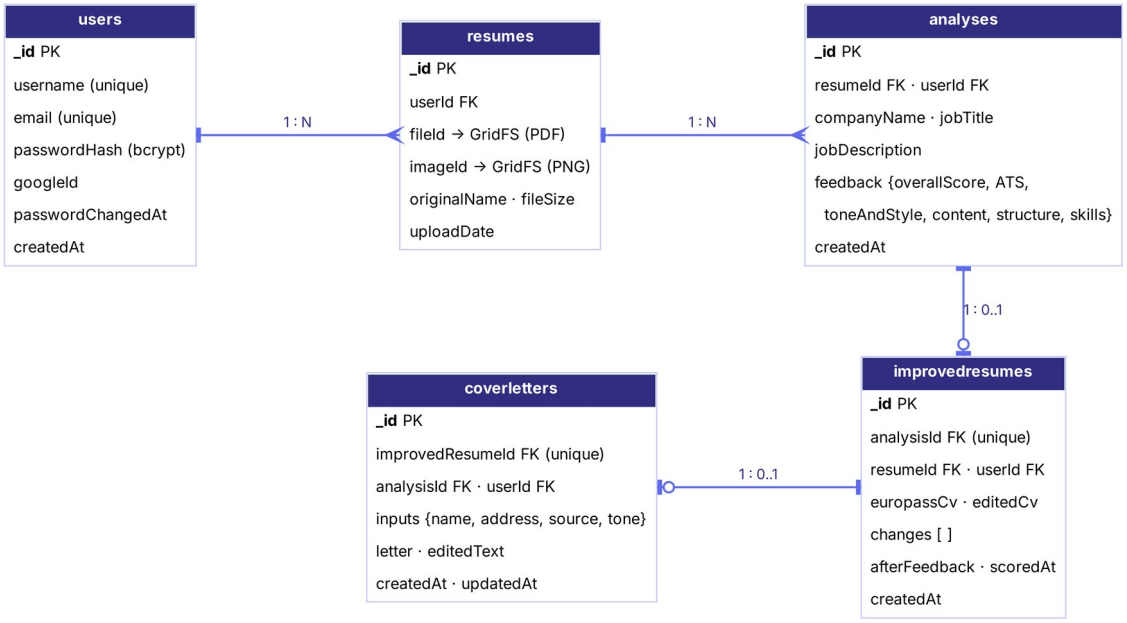


Figure 2: Entity–relationship diagram of the final database

Table 6: Collections

Collection	Key fields	Relationship
users	username (unique), email (unique, lowercase), passwordHash, hasPassword, googleId (unique, sparse), passwordChangedAt, createdAt	1 user → N resumes
resumes	userId, fileId, imageId (→ GridFS), originalName, fileSize, uploadDate	1 resume → N analyses
analyses	resumeId, userId, companyName, jobTitle, jobDescription, feedback { overallScore, ATS, toneAndStyle, content, structure, skills }, createdAt	1 analysis → 0..1 improved resume
improvedresumes	analysisId (unique), resumeId, userId, europassCv, editedCv, editedAt, changes[], afterFeedback, scoredAt	1 improved resume → 0..1 cover letter
coverletters	improvedResumeId (unique), analysisId (unique), userId, inputs, letter, editedText, timestamps	—
uploads.files / .chunks	GridFS: the PDF and PNG bytes in 255 KB chunks	referenced by resumes

6.1 Design decisions

- **Ownership on every document:** `userId` is stored on analyses, improved resumes and cover letters, so every query is filtered by the logged-in user and one user can never read another user's data.

- **Unique indexes enforce cardinality:** one improved resume per analysis and one cover letter per improved resume; regenerating replaces the previous version.
- **Cascade delete:** deleting a resume removes its analyses, improved resumes, cover letters and both GridFS files; deleting the account removes everything owned by the user.
- **AI output kept separate from user edits:** `europassCv` (AI) and `editedCv` (user) are stored side by side, so the user can always reset to the AI version.
- **Files in GridFS, not on disk:** Render's disk is temporary, so storing files in Atlas keeps them safe across redeploys.

7. AI Integration

All four AI features share one service (`geminiService.callGeminiJson`) that sends a prompt to Gemini with `responseMimeType: "application/json"` and a `responseJsonSchema`. The model is therefore forced to return valid JSON in a known shape; the backend still validates and normalises every field before saving.

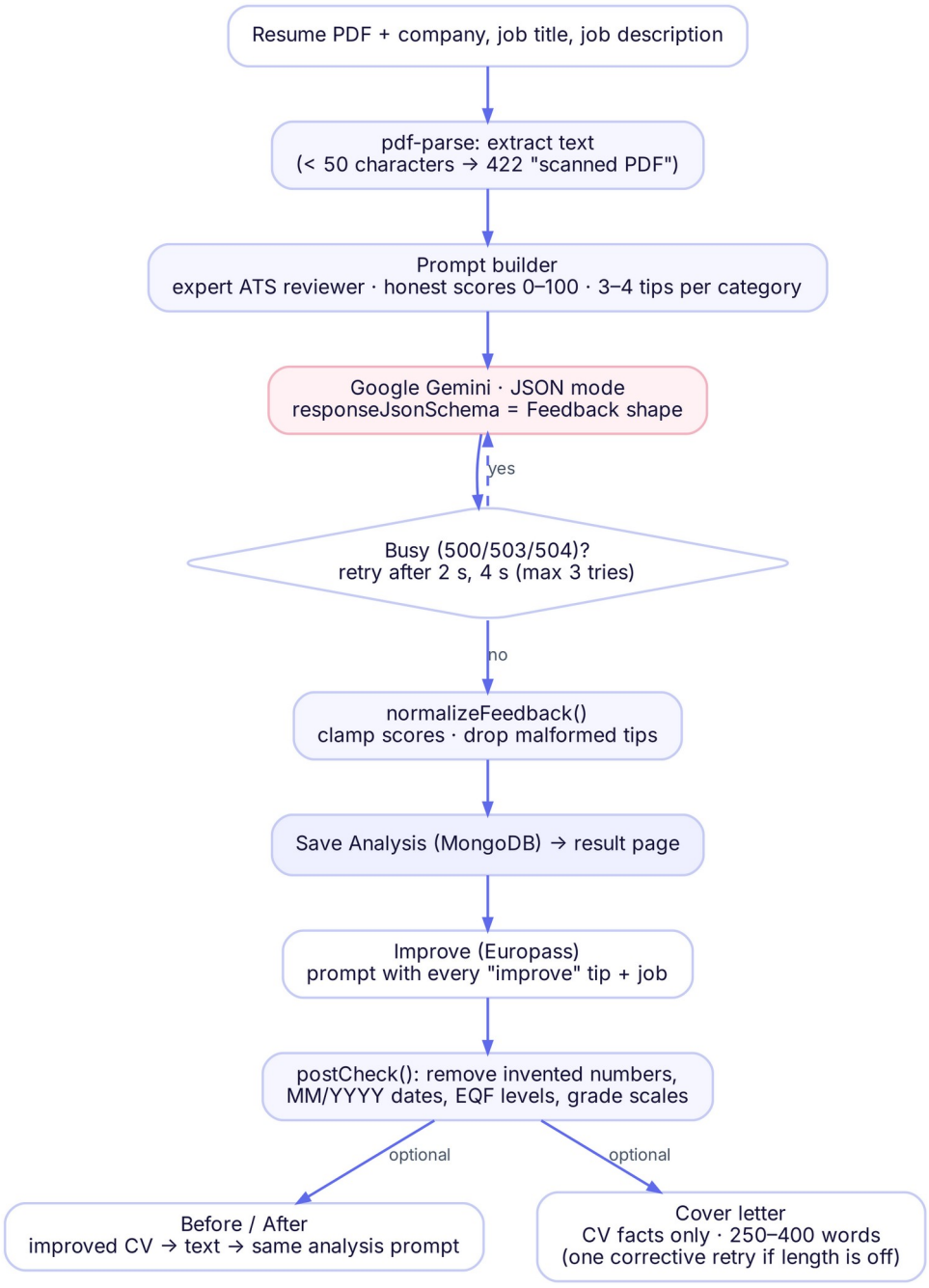


Figure 3: AI workflow from PDF upload to cover letter

7.1 Resume analysis

The resume text (up to 30 000 characters) is extracted with pdf-parse and sent together with the company, job title and job description. The schema requires an `overallScore` (0–100) and five categories, each with a `score` and a list of tips of type `good` or `improve` with an explanation. Scores are clamped to 0–100 and malformed tips are dropped.

```
{ "overallScore": 72,
  "ATS":          { "score": 78, "tips": [ { "type": "good",    "tip": "Clear section
headings" } ] },
  "toneAndStyle": { "score": 75, "tips": [ { "type": "improve", "tip": "Avoid first
person", "explanation": "..." } ] },
  "content": { ... }, "structure": { ... }, "skills": { ... } }
```

7.2 Europass CV improvement

The improvement step sends the resume text, every "improve" tip and the job details. The prompt sets strict rules: never invent experience, degrees, companies, dates, numbers or skills; keep job titles exactly; rephrase with strong action verbs; add job keywords only where the candidate really has the skill; use clear placeholders such as `[Add your English CEFR level, e.g. B2]` where Europass needs missing information; and list every change with its reason.

Because a language model can still ignore instructions, a deterministic **post-check** in code enforces the most important rules after Gemini answers:

Table 7: Post-check rules applied in code

Rule	Example
Numbers not present in the original resume become placeholders	"by 40%" → "by [Add a measurable result, e.g. X %]"
All dates in MM/YYYY format	"November 30, 2024" → "11/2024"
Ongoing degree shown as expected graduation	"Current" → "Expected 08/2026"
Qualification names written in full with EQF level	"H.S.C" → "Higher Secondary Certificate (HSC)", EQF 4
Grade with its scale	"4.67" → "GPA 4.67 / 5.00"
Personal data not used in Europass removed	religion, marital status, parents' names, date of birth

7.3 Before / after score

The improved CV (the user's edited version if one exists) is converted back to text, placeholders are removed so that only real content counts, and it is scored with the same prompt and the same job as the original analysis. The result is stored as `afterFeedback`; the result page shows the change per category and a score-history chart.

7.4 Cover letter

Gemini writes only the creative parts of the letter — the opening, two body paragraphs, the closing and a list of highlights that map job requirements to CV evidence. The backend fills in the fixed parts (sender, date, recipient, subject, salutation and sign-off) so that the format is always correct. If the body is outside 250–400 words, Gemini is asked once more to fix the length.

7.5 Reliability and cost control

- **Retries:** temporary Gemini errors (500/503/504) are retried up to three times with 2 s and 4 s pauses.
- **Clear errors:** 422 for scanned PDFs without text, 429 for rate limits, 502 for model failure, 503 when the key is missing or the model is still busy.
- **Quota:** each user may make 15 AI requests per hour; failed requests do not count.
- **Key safety:** the Gemini API key exists only in the server environment and is never sent to the browser.

8. Security and Authentication

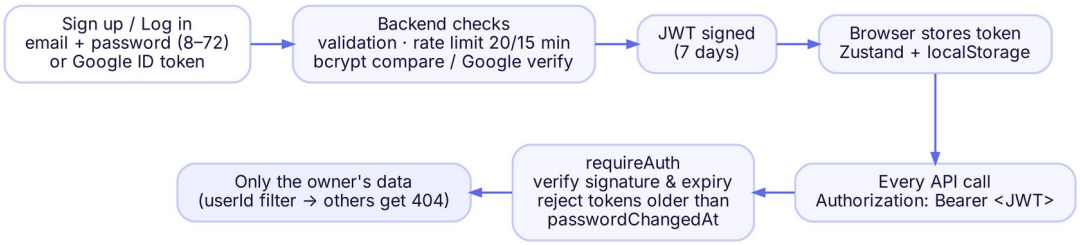


Figure 4: Authentication flow (email/password and Google Sign-In)

- **Passwords** are 8–72 characters and hashed with bcrypt; the plain password is never stored or logged.
- **JWT** tokens are signed with a long random secret and expire after 7 days. When the password changes, `passwordChangedAt` is updated and every older token is rejected, which logs out other devices.
- **Google Sign-In** sends a Google ID token to the backend, which verifies it with google-auth-library against the project's client id and only accepts verified e-mail addresses.
- **Rate limits:** 20 auth requests per 15 minutes per IP (register, login, Google, change password, delete account) and 15 AI requests per hour per user.
- **Input validation:** every body and URL parameter is checked with express-validator; ids must be valid ObjectIds and belong to the user.

- **Uploads:** PDF only, max 10 MB, magic-byte check (`%PDF-`), random file names, stored in GridFS.
- **HTTP hardening:** helmet security headers, CORS restricted to the frontend origin, 1 MB JSON limit, central error handler that never leaks stack traces in production.
- **Secrets:** stored in `.env` locally and in the Vercel/Render dashboards in production; `.env` is git-ignored.

9. Implementation

The frontend has nine routes. Logged-out visitors see the landing page; after login they go straight to the dashboard. Every logged-in page shares the same top bar, page title and card style, and a progress stepper (Upload → Analysis → Europass CV → Cover letter) shows where the user is.

Table 8: Frontend routes

Route	Page	Purpose
`/`	Landing	Product introduction, how it works, features, team
`/auth`	Login / Sign up	Email + password or Google Sign-In
`/dashboard`	Dashboard	History of resumes and analyses, score badges
`/upload`	Upload	PDF drop zone, job fields, processing animation
`/resume/:id`	Result	Scores, tips, before/after, history chart, report PDF
`/resume/:id/improved/:analysisId`	Europass CV	Improved CV, inline editing, change list, PDF
`.../cover-letter`	Cover letter	Generate, edit and download the letter
`/account`	Account	Change password, delete account
`/wipe`	Wipe	Delete all of the user's data

9.1 Landing and authentication

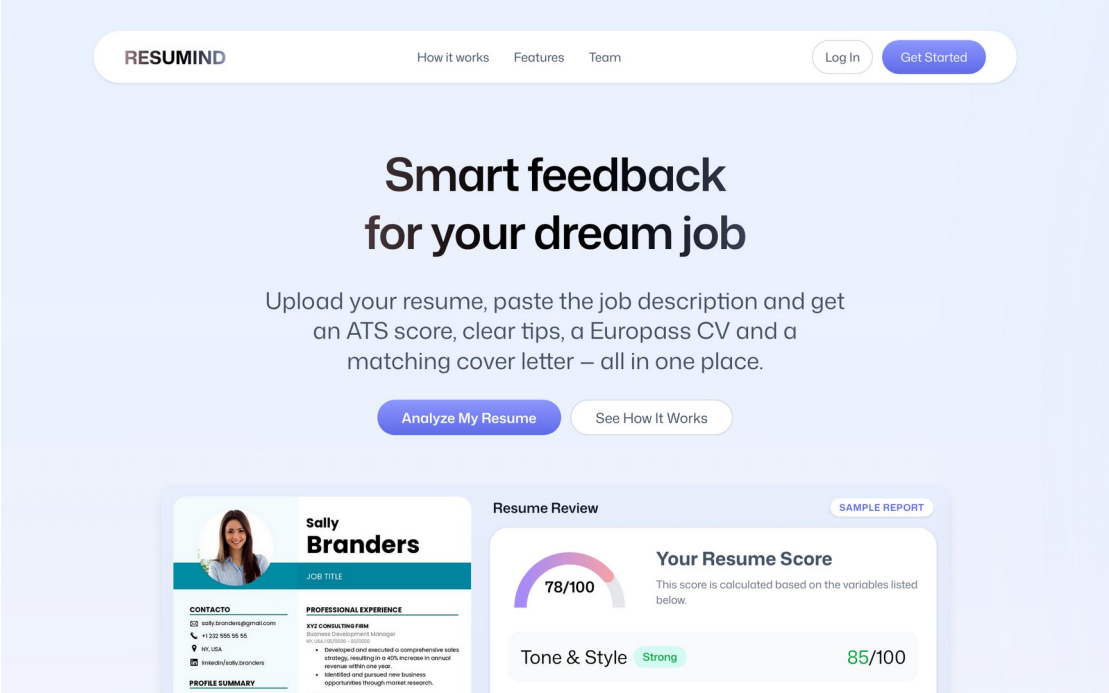


Figure 5: Landing page

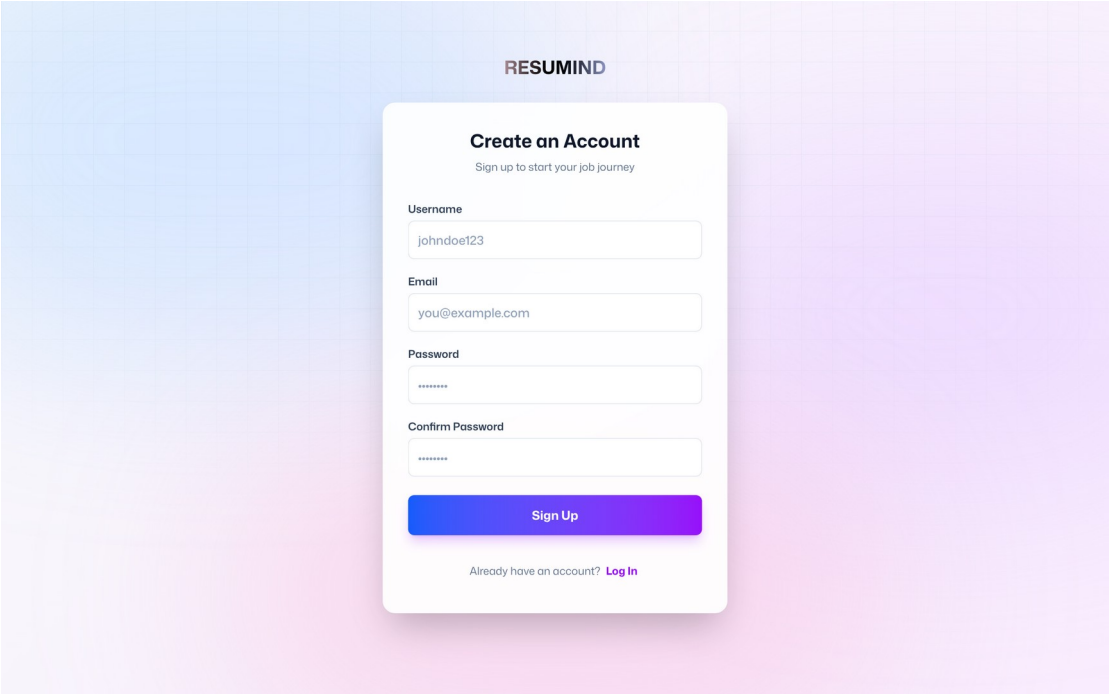


Figure 6: Sign-up dialog with Google Sign-In

9.2 Dashboard and upload

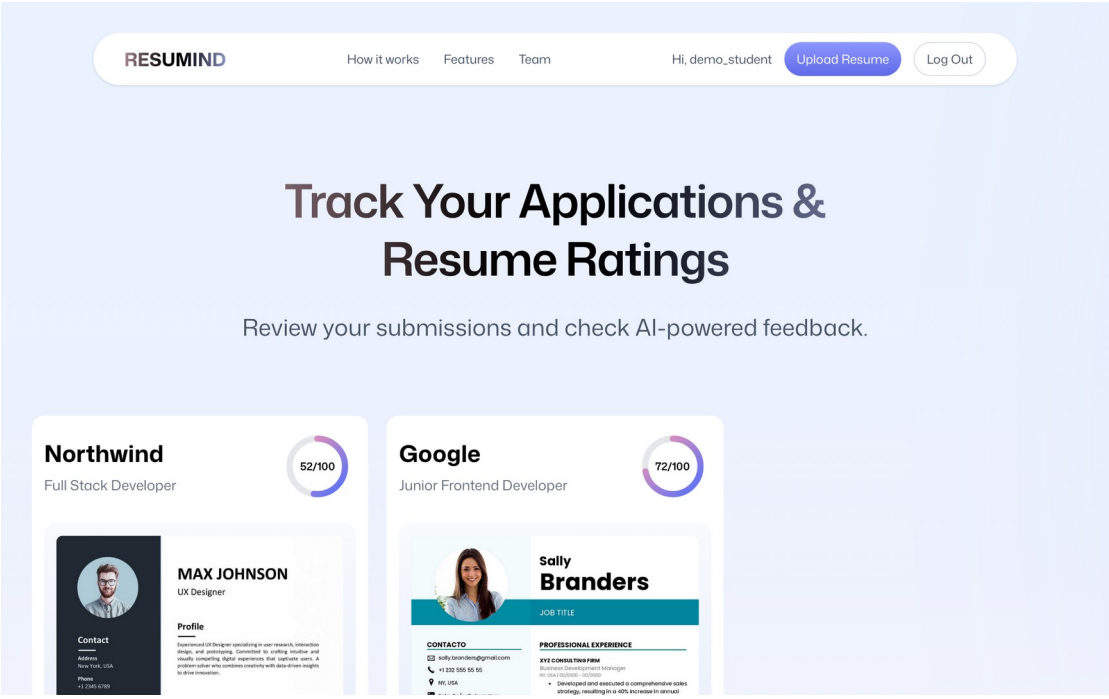


Figure 7: Dashboard with resume history

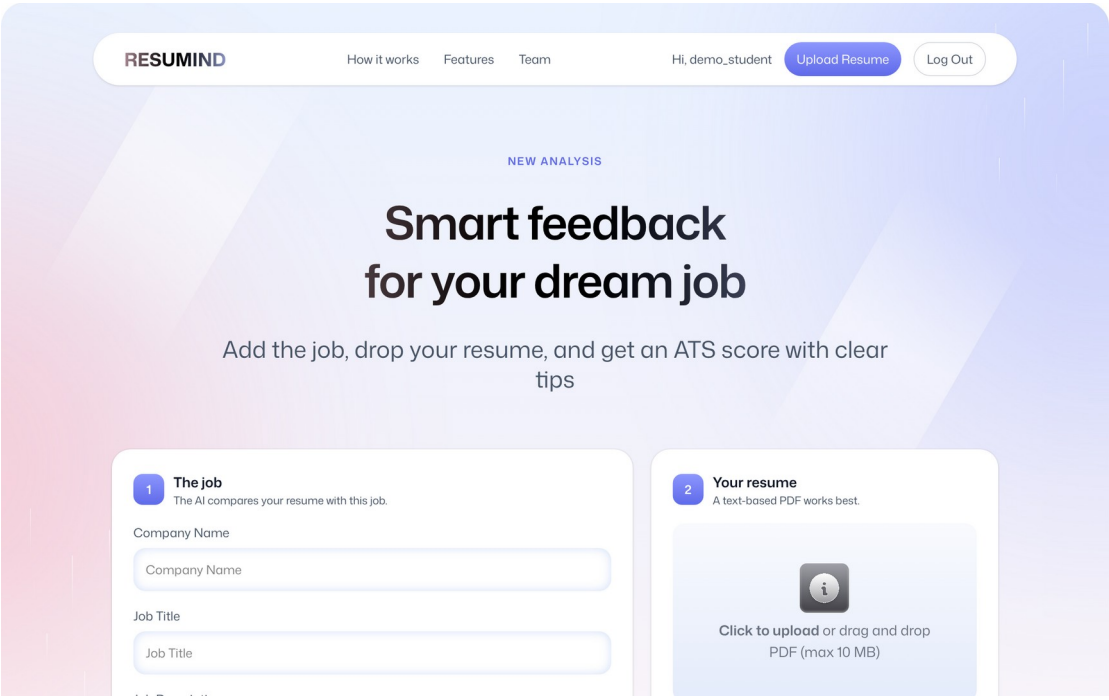


Figure 8: Upload page with job details

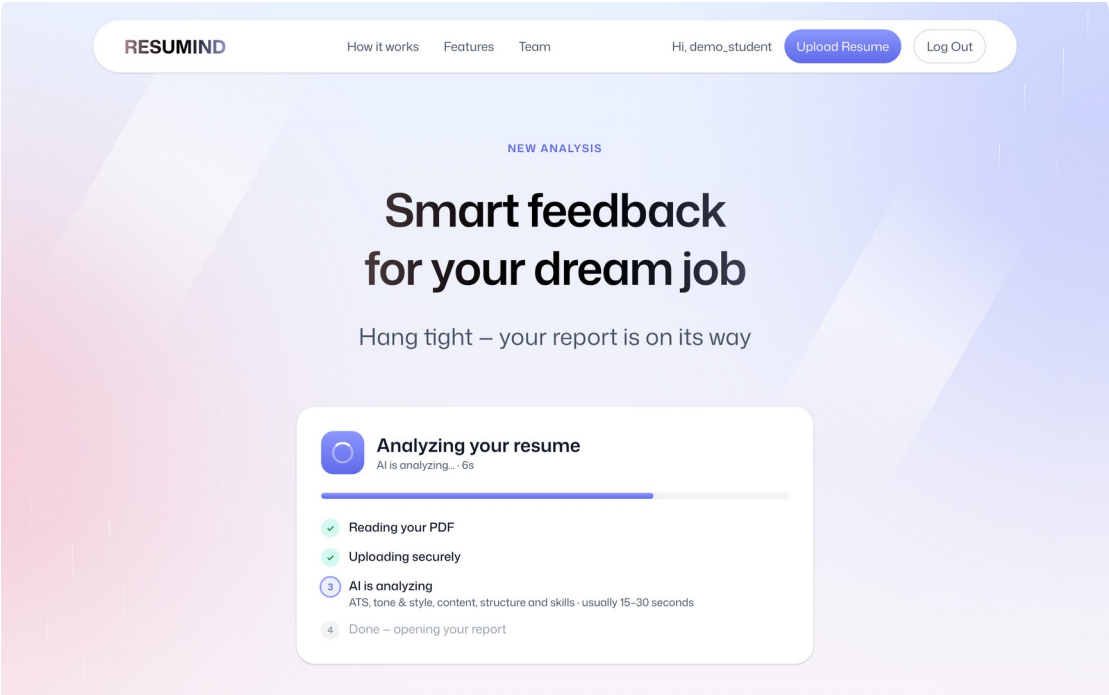


Figure 9: Processing state while Gemini analyses the resume

9.3 Analysis result

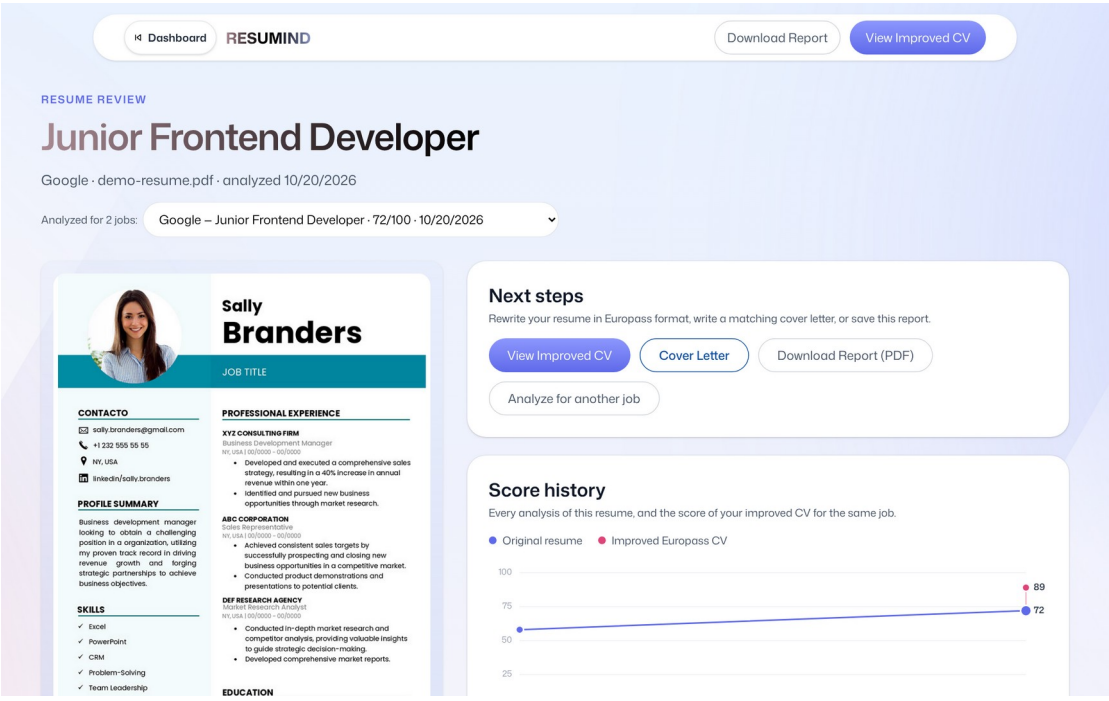


Figure 10: Result page: overall score and category scores

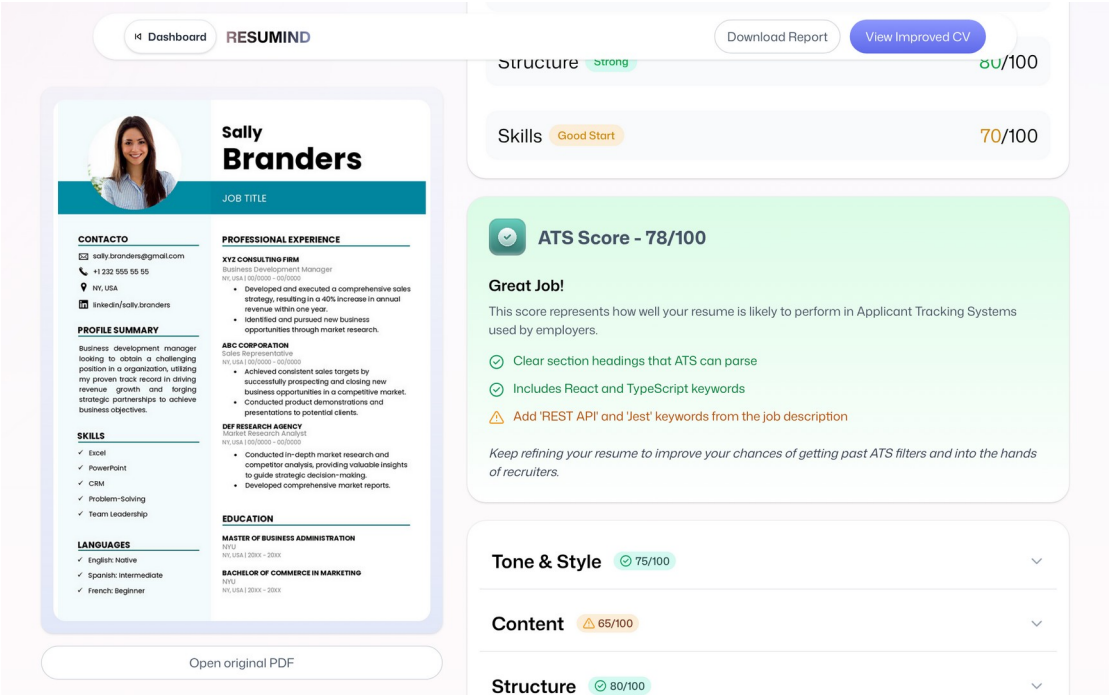


Figure 11: Detailed tips per category

9.4 Europass CV and cover letter

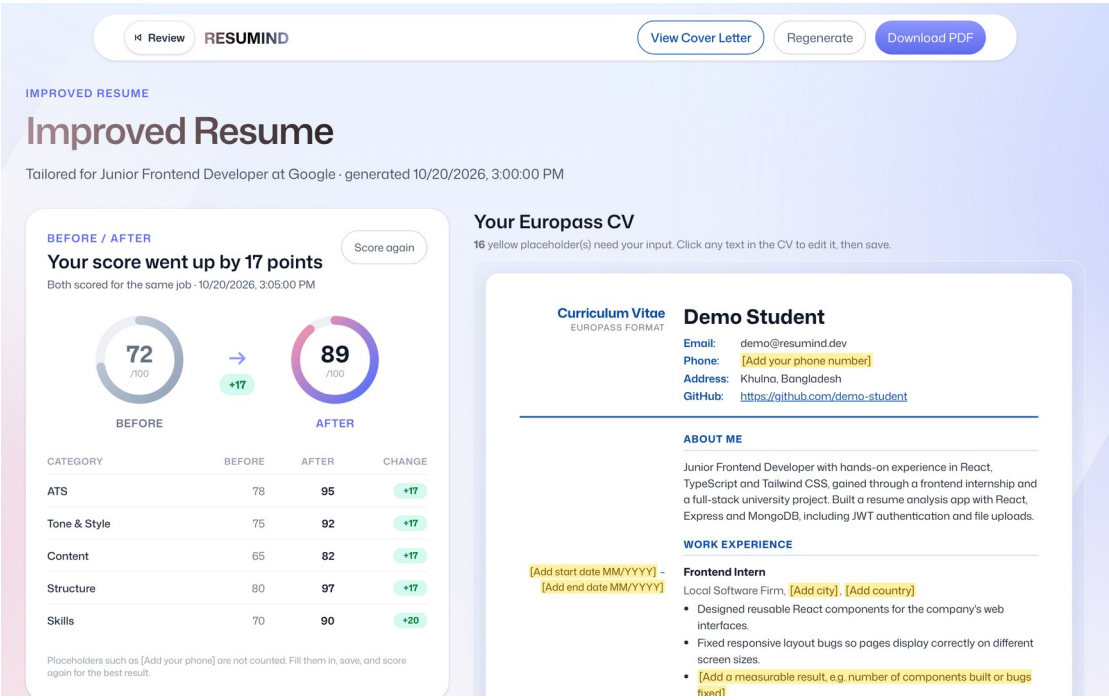


Figure 12: Improved Europass CV with change list

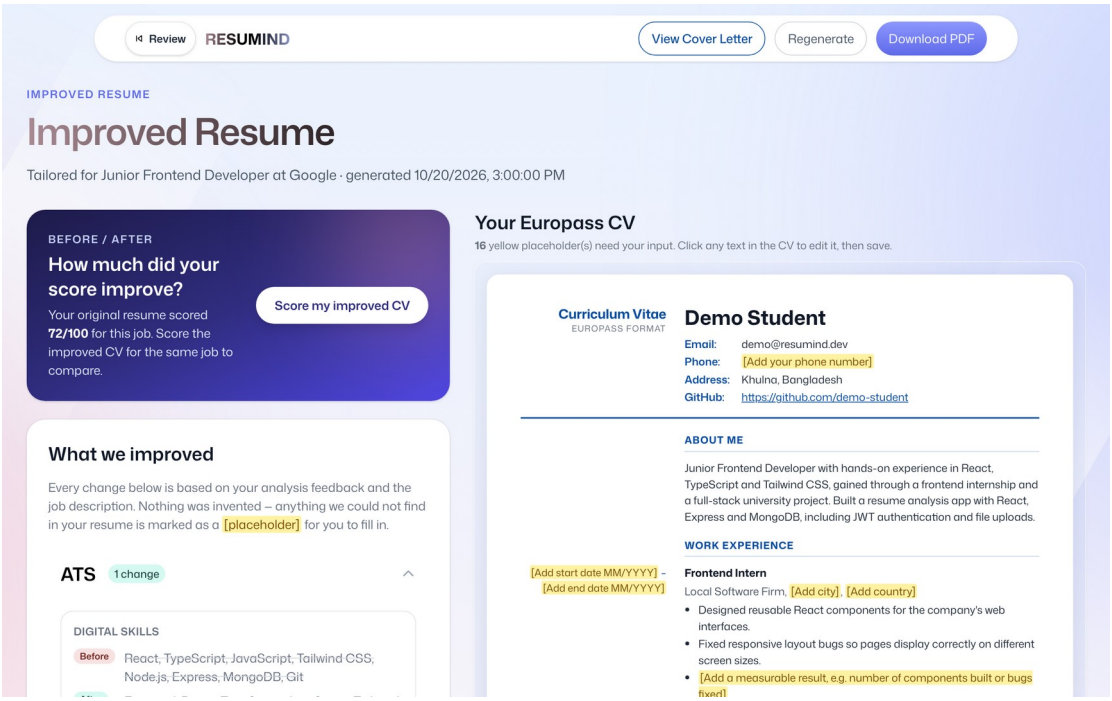


Figure 13: Before / after score comparison

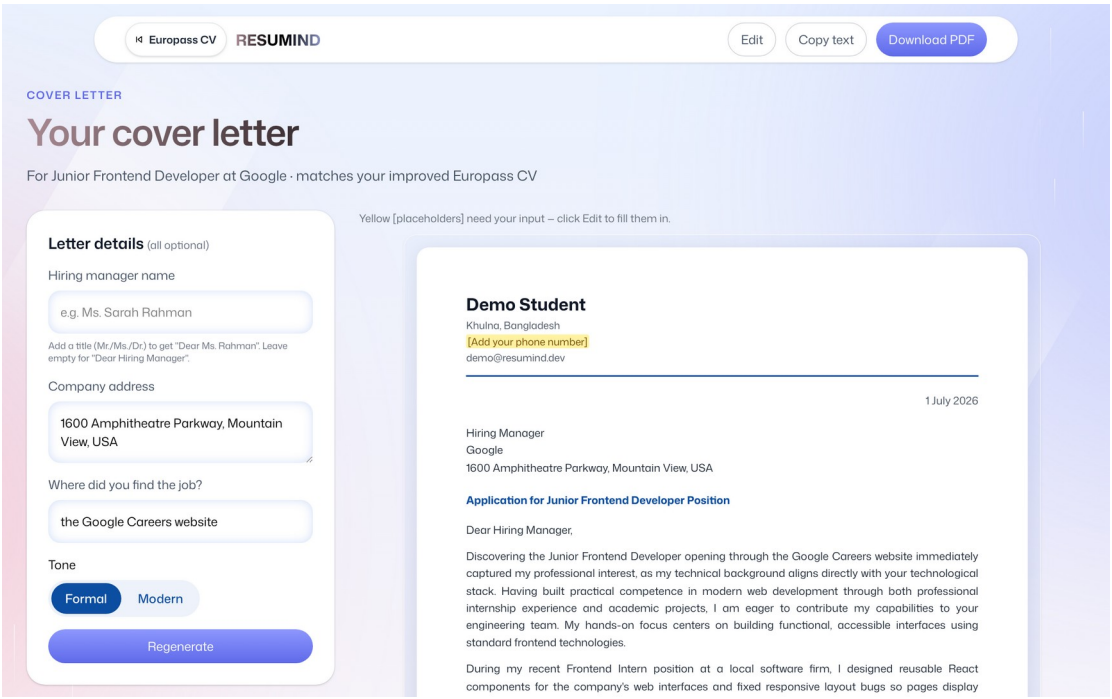


Figure 14: Generated cover letter

9.5 Responsive design

The interface is mobile-first. On phones the result page stacks the feedback above the preview, navigation collapses and all buttons remain within thumb reach; on desktop the layout uses multiple columns.

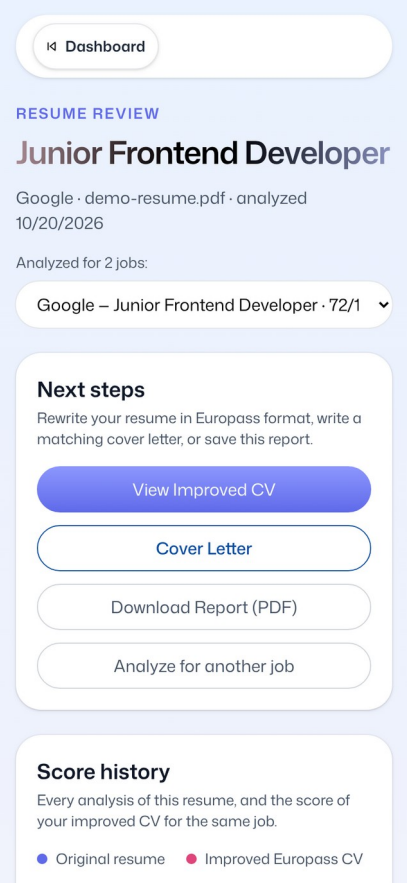


Figure 15: Result page on a phone

10. Testing

10.1 Automated API tests

The backend is tested with **Vitest** and **Supertest**, which send real HTTP requests to the Express app. Each test run starts a fresh in-memory MongoDB (mongodb-memory-server) and replaces Gemini with a mock, so the tests are fast, repeatable, need no API key and never touch real data.

Table 9: Automated test suites

Suite	Tests	What is covered
auth.test.js	10	Register, login, validation, duplicate e-mail/username, /me, password change (old token rejected), delete account
google.test.js	9	Google Sign-In: new and existing users, account linking, unverified e-mail, invalid token, Google-only password rules
resume.test.js	12	Upload (PDF only, fake PDF rejected, size), list, get, file/image download, ownership, cascade delete
analyze.test.js	4	Analysis with mocked Gemini, input validation, other user's resume, missing API key

Suite	Tests	What is covered
europass.test.js	13	Improve, post-check rules, edit/reset CV, before/after score, cover letter generate/edit/reset
Total	48	All passing in GitHub Actions

10.2 Continuous integration

A GitHub Actions workflow runs on every push and pull request. Job 1 installs the backend with `npm ci` and runs all 48 tests; job 2 type-checks and builds the frontend. A green tick next to a commit means both jobs passed; Vercel and Render then deploy the same commit.

10.3 Manual and device testing

Table 10: Manual test cases (selection)

ID	Scenario	Expected result	Result
M1	Sign up, log out, log in again	Dashboard opens; logout returns to the landing page	Pass
M2	Upload a scanned (image-only) PDF	Clear message: the PDF has no readable text	Pass
M3	Upload a .docx renamed to .pdf	Rejected: not a real PDF	Pass
M4	Analyse a CV for a frontend job	Scores and tips shown in about 10–20 s	Pass
M5	Improve to Europass, edit a line, reload	Edit is kept; reset restores the AI version	Pass
M6	Score the improved CV	Before/after comparison and chart update	Pass
M7	Generate a cover letter and download PDF	One A4 page, correct header and salutation	Pass
M8	Open the app on a phone (360 px)	No horizontal scrolling; all actions reachable	Pass
M9	Visit an unknown URL	Friendly 404 page with a way back	Pass
M10	Change password on one device	Other devices are logged out	Pass

11. Results and Discussion

All planned features are implemented, deployed and reachable from a single public URL. In the team's own trials with real student resumes, the analysis typically returned in 10–20 seconds, the Europass rewrite in 20–40 seconds and the cover letter in under 20 seconds on the free Gemini tier.

- **Explainable feedback:** every score comes with "good" and "improve" tips and an explanation, so users know what to change and why.
- **Measured improvement:** the before/after score turns the rewrite into a visible result; in the sample data the overall score rose after the Europass rewrite while placeholders kept the CV honest.
- **Truthfulness:** the post-check prevents invented numbers and dates, and placeholders clearly show the user what to fill in.
- **Usability:** a single flow from upload to cover letter with a progress stepper; mobile layout tested on real phones.
- **Reliability:** 48 automated tests and CI on every push; automatic retries reduce failures when Gemini is busy.

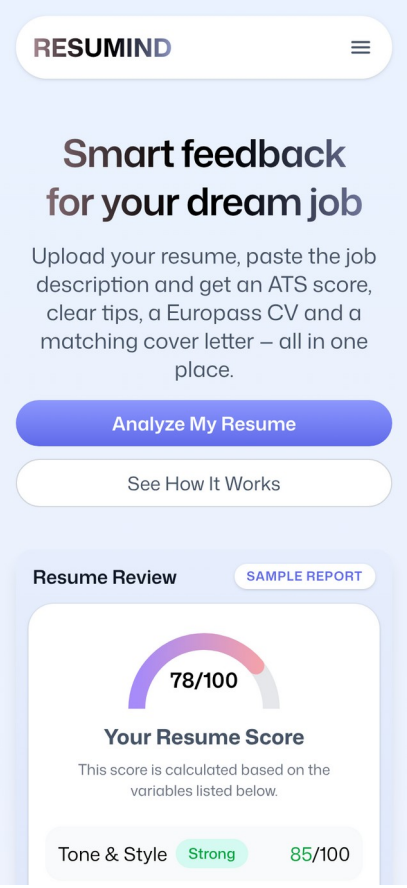


Figure 16: Landing page on a phone

12. Deployment

Table 11: Production environment

Part	Platform	Details
Frontend	Vercel	https://ai-resume-analyzer-psi-lime.vercel.app Static SPA build of React Router; VITE_API_URL and VITE_GOOGLE_CLIENT_ID set in the dashboard

Part	Platform	Details
Backend	Render (free web service)	https://resumind-api-3kpd.onrender.com Blueprint render.yaml, npm ci --omit=dev, health check /api/health, auto-deploy on push
Database	MongoDB Atlas (M0)	Collections + GridFS; network access 0.0.0.0/0 because Render has no fixed IP
Secrets	.env / dashboards	MONGO_URI, JWT_SECRET (generated by Render), GEMINI_API_KEY, GOOGLE_CLIENT_ID, CLIENT_URL — never committed

Operational note: The free Render instance sleeps after 15 minutes without traffic, so the first request afterwards can take about 50 seconds. Before a demonstration the team opens /api/health once to wake the server.

13. Limitations and Future Work

13.1 Current limitations

- Only text-based PDF resumes are supported; scanned images need OCR.
- AI scores can vary slightly between runs because they come from a language model.
- The free Gemini and Render tiers limit throughput and cause a cold start after inactivity.
- The interface and AI output are in English only.

13.2 Future work

- OCR (e.g. Tesseract) for scanned resumes and support for DOCX uploads.
- Job search integration: paste a LinkedIn or BDJobs link instead of the description.
- Interview preparation: AI-generated questions based on the CV and the job.
- Bangla language support for the interface and feedback.
- Native mobile app (React Native) sharing the same REST API, with push notifications.
- Export the Europass CV as the official Europass XML so it can be imported into the EU portal.
- Recruiter mode: rank several resumes against one job description.

14. Conclusion

RESUMIND shows how a carefully constrained large language model can turn a stressful, opaque step of job hunting into a guided, explainable process. Starting from a tutorial prototype, the team built a complete three-tier system with its own secure backend, a database that follows the course ERD, four AI features with structured output and code-level safeguards,

automated testing with continuous integration, and a public deployment that works on phones and desktops. The project met all objectives of the proposal and gave every team member practical experience with full-stack development, AI integration, security and teamwork.

15. Team Contributions

Table 12: Roles and responsibilities

Member	ID	Role	Main responsibilities
Mahammad Hasan	11220320828	Team Leader · Frontend Developer + UI/UX Designer	Team coordination; React Router 8 app structure, Zustand auth store, API client and protected routes; all pages (auth, upload, dashboard, result, Europass CV editor, cover letter, account, wipe); responsive Tailwind CSS 4 UI/UX
Shanawaz Sakib	11220320916	Core System Integration Engineer	Google Gemini integration: JSON-mode prompts and schemas for analysis, Europass rewrite and cover letter, retries; end-to-end flow PDF → text → Gemini → saved feedback → UI; AI rate limit; Docker Compose setup
Ashikur Rahman Himel	11220320833	Database Storage & Quality Controller	MongoDB Atlas schema and Mongoose models (ERD, schema docs, seed data); file storage and input validation; automated API tests (Vitest + Supertest); dependency security audit
Rasel Ratul	11220320827	Backend Developer	Express 5 REST API: JWT authentication, Multer upload, controllers, routes, error handling; bcrypt, helmet, CORS, login rate limiting, ownership and PDF magic-byte checks; change-password and delete-account endpoints; Dockerfile

References

1. Europass — European Union, "Create your Europass CV", https://europass.europa.eu
2. Council of Europe, "Common European Framework of Reference for Languages (CEFR)".
3. European Commission, "The European Qualifications Framework (EQF)".
4. Google, "Gemini API — Structured output", https://ai.google.dev

5. MongoDB, "GridFS" documentation, https://www.mongodb.com/docs
6. OWASP Foundation, "Authentication Cheat Sheet" and "Password Storage Cheat Sheet".
7. React Router documentation, https://reactrouter.com
8. Express.js documentation, https://expressjs.com
9. JavaScript Mastery, "Build an AI Resume Analyzer" (YouTube tutorial used for the first prototype).
10. Mobile Computing Lab Student Handbook, CSE4204, NUBTK.